

TJSC 2026 - Analista de Sistemas

Banco de Dados e Engenharia de Dados

Aula 4

SQL parte 1 - SELECT, WHERE, JOIN, GROUP BY, HAVING, ORDER BY, funções e subconsultas

Material original, orientado ao edital TJSC 2026 e calibrado para o padrão FGV. Aula focada em consulta SQL, leitura de resultados, raciocínio sobre junções e agregações, com 20 questões autorais comentadas.

Disciplina	Banco de Dados e Engenharia de Dados
Aula	4 de 12
Banca	FGV - Fundação Getulio Vargas
Recorte da trilha	SQL parte 1: SELECT, WHERE, JOIN, GROUP BY, HAVING, ORDER BY, funções e subconsultas
Peso estratégico	SQL costuma render questão direta ou questão misturada com modelagem, relatórios e administração

1. Direcionamento da aula

Esta aula cobre a parte mais operacional da Linguagem SQL - Structured Query Language - exigida no edital: consultas com SELECT, filtros com WHERE, junções com JOIN, agregações com GROUP BY e HAVING, ordenação com ORDER BY, funções comuns e subconsultas. O objetivo é transformar experiência prática com bancos e APIs em acerto de prova FGV.

No edital do TJSC, Banco de Dados e Engenharia de Dados aparece como disciplina de 8 questões e inclui expressamente Linguagem SQL. Portanto, esta aula não é um curso enciclopédico de SQL: é uma preparação direcionada para interpretar consultas, identificar erros semânticos, prever resultados e evitar confusões clássicas entre WHERE e HAVING, INNER JOIN e LEFT JOIN, COUNT(*) e COUNT(coluna), IN e EXISTS, DISTINCT e GROUP BY.

Meta de prova: sair capaz de resolver questões em que a FGV apresenta tabelas, scripts SQL e um requisito de relatório, exigindo a consulta correta ou o resultado produzido.

2. O que a FGV tende a cobrar em SQL

A FGV costuma cobrar SQL em formato aplicado: uma tabela de processo, pedido, cliente, servidor ou documento; um requisito de relatório; e alternativas com consultas muito próximas. O erro não costuma ser desconhecer a palavra SELECT, mas confundir a etapa lógica da consulta, colocar agregação no WHERE, usar LEFT JOIN e depois filtrá-lo como INNER JOIN, esquecer colunas no GROUP BY ou supor ordenação sem ORDER BY.

Padrão de cobrança	Como aparece	Armadilha provável
Consulta com requisito de negócio	Relatório de processos, clientes, pedidos, partes ou usuários	Trocar filtro de linha por filtro de grupo
Leitura de resultado	Tabelas pequenas e pergunta sobre linhas retornadas	Ignorar duplicação causada por JOIN
Agregação	COUNT, SUM, AVG, MIN, MAX com GROUP BY	Usar WHERE com função agregada
Subconsulta	IN, EXISTS, NOT EXISTS, subconsulta correlacionada	Cair no problema de NOT IN com NULL
Dialeto de SGBD	LIMIT/OFFSET, TOP, FETCH FIRST	Misturar sintaxe de MySQL/PostgreSQL com SQL Server ou Oracle

3. Ordem sintática e ordem lógica

A ordem em que escrevemos uma consulta não é exatamente a ordem lógica em que ela é processada. Em prova, isso explica por que WHERE não pode filtrar SUM(valor) e por que aliases definidos no SELECT podem ou não estar disponíveis em determinadas cláusulas, dependendo do SGBD.

Ordem sintática comum:
 SELECT ...
 FROM ...
 JOIN ... ON ...
 WHERE ...
 GROUP BY ...
 HAVING ...
 ORDER BY ...

Ordem lógica simplificada:
 1. FROM / JOIN
 2. ON
 3. WHERE
 4. GROUP BY
 5. HAVING
 6. SELECT
 7. DISTINCT
 8. ORDER BY

Pegadinha central: WHERE filtra linhas antes da agregação. HAVING filtra grupos depois da agregação.

4. SELECT: projeção, aliases e DISTINCT

4.1 SELECT não é filtro de linhas

O SELECT escolhe as expressões que aparecerão no resultado. Na álgebra relacional, isso se aproxima da projeção: selecionar colunas ou expressões. O filtro de linhas fica no WHERE; a combinação de tabelas fica no FROM/JOIN; o agrupamento fica no GROUP BY.

```
SELECT numero_processo, classe, data_distribuicao
FROM processo;
```

A consulta acima não filtra processos: ela apenas escolhe três colunas da relação PROCESSO.

4.2 Alias

Alias é um apelido para coluna, expressão ou tabela. Em prova, alias melhora legibilidade e evita ambiguidade em JOIN.

```
SELECT p.numero AS numero_processo,
       u.nome AS unidade_responsavel
FROM processo p
JOIN unidade u ON u.id_unidade = p.id_unidade;
```

4.3 DISTINCT

DISTINCT remove duplicatas do conjunto de expressões selecionadas. Ele não remove duplicatas da tabela original inteira se você seleciona apenas algumas colunas. A duplicidade é avaliada sobre a lista do SELECT.

```
SELECT DISTINCT cidade
FROM pessoa;

SELECT DISTINCT cidade, estado
FROM pessoa;
```

A primeira consulta retorna cidades distintas. A segunda retorna pares distintos de cidade e estado. Isso é diferente de uma linha por pessoa.

5. FROM e JOIN

5.1 FROM

A cláusula FROM define a origem dos dados. Quando há mais de uma tabela, é preciso combinar essas fontes. Se a consulta lista várias tabelas sem condição, o resultado lógico tende ao produto cartesiano, multiplicando linhas.

```
SELECT *
FROM processo, parte; -- sem condição de junção: tende a produto cartesiano
```

Em sistemas reais, isso costuma gerar resultado enorme e incorreto. Em prova FGV, pode aparecer como pegadinha de contagem de linhas.

5.2 INNER JOIN

INNER JOIN retorna apenas linhas que possuem correspondência segundo a condição de junção. É o join natural para processos com movimentações, clientes com pedidos ou servidores com lotação registrada quando se deseja apenas correspondências.

```
SELECT p.numero, m.data_movimentacao
FROM processo p
JOIN movimentacao m ON m.id_processo = p.id_processo;
```

5.3 LEFT JOIN

LEFT JOIN preserva todas as linhas da tabela à esquerda e traz dados da direita quando houver correspondência. Quando não houver, as colunas da direita aparecem como NULL. É a escolha clássica para procurar registros sem correspondência, usando IS NULL.

```
SELECT p.numero
FROM processo p
LEFT JOIN audiencia a ON a.id_processo = p.id_processo
WHERE a.id_audiencia IS NULL;
```

Essa consulta retorna processos sem audiência associada. Esse padrão aparece em prova porque combina JOIN, NULL e WHERE.

5.4 Cuidado: filtro no WHERE pode destruir o LEFT JOIN

```
SELECT p.numero, a.data_audiencia
FROM processo p
LEFT JOIN audiencia a ON a.id_processo = p.id_processo
WHERE a.situacao = 'MARCADA';
```

Embora a consulta use LEFT JOIN, o filtro no WHERE exige valor na tabela da direita, eliminando os processos sem audiência. Na prática, isso se comporta como INNER JOIN para esse filtro. Se a intenção é preservar processos sem audiência ou filtrar a correspondência sem eliminar a esquerda, a condição deve ser pensada no ON ou combinada com IS NULL, conforme o requisito.

6. WHERE: filtro de linhas, NULL e operadores

6.1 WHERE

WHERE filtra linhas antes do agrupamento. Pode usar comparações, operadores lógicos, BETWEEN, LIKE, IN, EXISTS e predicados de nulidade.

```
SELECT numero, classe
FROM processo
WHERE classe = 'CIVEL'
AND data_distribuicao >= DATE '2026-01-01';
```

6.2 NULL não é valor comum

NULL representa ausência, desconhecimento ou inaplicabilidade de valor. Comparações com NULL não retornam verdadeiro; por isso, usa-se IS NULL ou IS NOT NULL.

```
WHERE data_baixa = NULL -- errado em SQL usual
WHERE data_baixa IS NULL -- correto
```

Pegadinha: WHERE status <> CANCELADO não inclui automaticamente linhas com status NULL. Para incluir NULL, seria necessário explicitar: status <> CANCELADO OR status IS NULL.

6.3 BETWEEN, LIKE e IN

Operador	Uso típico	Cuidado de prova
BETWEEN	Faixa entre limites	Em SQL, é inclusivo nos limites
LIKE	Busca por padrão textual	% representa qualquer sequência; _ representa um caractere
IN	Pertencimento a lista ou subconsulta	NOT IN com NULL pode derrubar o resultado esperado
EXISTS	Existência de ao menos uma linha em subconsulta	Não importa a lista selecionada; importa existir linha

7. GROUP BY e funções de agregação

7.1 Para que serve GROUP BY

GROUP BY agrupa linhas que possuem os mesmos valores nas expressões indicadas. Normalmente aparece com funções agregadas, como COUNT, SUM, AVG, MIN e MAX.

```
SELECT p.id_unidade, COUNT(*) AS total_processos
FROM processo p
GROUP BY p.id_unidade;
```

7.2 Regra das colunas não agregadas

Em SQL relacional estrito, se uma coluna aparece no SELECT sem agregação, ela deve estar no GROUP BY, salvo exceções de dependência funcional ou comportamentos específicos do SGBD. Para prova FGV, assuma a regra conservadora: coluna não agregada no SELECT deve constar no GROUP BY.

```
SELECT u.nome, COUNT(*) AS total
FROM unidade u
JOIN processo p ON p.id_unidade = u.id_unidade
GROUP BY u.nome;
```

7.3 Funções agregadas

Função	Resultado	Pegadinha
COUNT(*)	Conta linhas	Conta linha mesmo que colunas tenham NULL
COUNT(coluna)	Conta valores não nulos da coluna	Ignora NULL
SUM(coluna)	Soma valores não nulos	Retorna NULL se não houver valores em alguns SGBDs/cenários
AVG(coluna)	Média dos valores não nulos	Não divide por linhas com NULL
MIN/MAX	Menor/maior valor não nulo	Podem operar sobre datas e textos conforme ordenação

8. HAVING

HAVING filtra grupos. Ele é avaliado depois do GROUP BY. A forma mais comum de cair é um relatório que pede somente unidades com mais de 100 processos ou clientes cujo total gasto ultrapassa R\$ 1.000.

```
SELECT u.nome, COUNT(*) AS total_processos
FROM unidade u
JOIN processo p ON p.id_unidade = u.id_unidade
WHERE p.situacao = 'ATIVO'
GROUP BY u.nome
HAVING COUNT(*) > 100
ORDER BY total_processos DESC;
```

WHERE filtrou linhas de processos ativos antes do agrupamento. HAVING filtrou grupos de unidades com mais de 100 processos depois do agrupamento.

9. ORDER BY

ORDER BY define a ordem do resultado. Sem ORDER BY, não há garantia lógica de ordenação, mesmo que o banco aparentemente retorne na ordem do índice, da inserção ou da chave primária.

```
SELECT numero, data_distribuicao
FROM processo
ORDER BY data_distribuicao DESC, numero ASC;
```

ASC significa ascendente e DESC significa descendente. Em muitos SGBDs, o alias definido no SELECT pode ser usado no ORDER BY.

10. Funções escalares e funções agregadas

Funções escalares atuam linha a linha; funções agregadas reduzem várias linhas em um resultado por grupo ou por consulta. A FGV explora a confusão entre elas.

Tipo	Exemplos	Ideia
Escalar textual	UPPER, LOWER, SUBSTRING, TRIM, CONCAT	Transforma valor de uma linha
Escalar de nulo	COALESCE	Substitui primeiro valor não nulo
Escalar de data	EXTRACT, DATEPART, YEAR, MONTH	Extraí parte de data conforme SGBD
Agregada	COUNT, SUM, AVG, MIN, MAX	Resume conjunto de linhas

```
SELECT COALESCE(email, 'sem-email') AS contato
FROM pessoa;
```

```
SELECT id_unidade, COUNT(*) AS total
FROM processo
GROUP BY id_unidade;
```

11. Subconsultas

11.1 Conceito

Subconsulta é uma consulta dentro de outra consulta. Pode aparecer em WHERE, HAVING, SELECT ou FROM. Em prova, os formatos mais importantes são IN, EXISTS, NOT EXISTS, subconsulta escalar e subconsulta correlacionada.

11.2 IN

```
SELECT p.numero
FROM processo p
WHERE p.id_unidade IN (
  SELECT u.id_unidade
  FROM unidade u
  WHERE u.comarca = 'Florianopolis'
);
```

A subconsulta retorna um conjunto de valores; a consulta externa seleciona processos cuja unidade pertence a esse conjunto.

11.3 EXISTS e NOT EXISTS

```
SELECT p.numero
FROM processo p
WHERE EXISTS (
  SELECT 1
  FROM movimentacao m
  WHERE m.id_processo = p.id_processo
);
```

EXISTS testa se a subconsulta retorna ao menos uma linha. A lista do SELECT interno é irrelevante para a existência; por isso é comum usar SELECT 1.

11.4 Subconsulta correlacionada

A subconsulta é correlacionada quando usa valor da consulta externa. No exemplo anterior, `m.id_processo = p.id_processo` liga a subconsulta à linha corrente de PROCESSO.

11.5 NOT IN com NULL

NOT IN é perigoso quando a subconsulta pode retornar NULL. Em SQL, a lógica de três valores pode fazer a comparação não resultar verdadeira para linha alguma. Em prova, prefira NOT EXISTS quando o objetivo é não possui correspondência.

```
SELECT p.numero
FROM processo p
WHERE NOT EXISTS (
  SELECT 1
  FROM audiencia a
  WHERE a.id_processo = p.id_processo
);
```

12. Dialeto de SGBD: o que saber nesta aula

O edital cita MySQL, Microsoft SQL Server 2019, PostgreSQL 17+ e Oracle 23ai. A Aula 8 aprofundará cada SGBD, mas em SQL parte 1 vale conhecer diferenças recorrentes de prova.

Tema	PostgreSQL / MySQL	SQL Server	Oracle
Limitar linhas	LIMIT n OFFSET m	TOP n ou OFFSET/FETCH	FETCH FIRST n ROWS ONLY
Concatenação	no PostgreSQL; CONCAT também existe	+ ou CONCAT	ou CONCAT
Data atual	CURRENT_DATE / CURRENT_TIMESTAMP	GETDATE() / SYSDATETIME()	SYSDATE / CURRENT_DATE
Aspas	strings com aspas simples	strings com aspas simples	strings com aspas simples

Para a FGV, a prioridade é entender SQL lógico. Diferenças de dialeto entram quando o enunciado cita expressamente PostgreSQL, MySQL, SQL Server ou Oracle.

13. Checklist do que memorizar

- SELECT escolhe colunas/expressões; WHERE filtra linhas; HAVING filtra grupos.
- FROM/JOIN ocorre antes de WHERE; GROUP BY ocorre antes de HAVING; ORDER BY só ordena no final.
- INNER JOIN mantém apenas correspondências; LEFT JOIN preserva a tabela da esquerda.
- Filtro no WHERE sobre coluna da direita pode transformar um LEFT JOIN, na prática, em INNER JOIN.
- COUNT(*) conta linhas; COUNT(coluna) conta valores não nulos da coluna.
- Coluna não agregada no SELECT deve aparecer no GROUP BY, como regra segura de prova.
- Sem ORDER BY, não presume ordem.
- BETWEEN é inclusivo; LIKE usa % e _; NULL exige IS NULL ou IS NOT NULL.
- EXISTS testa existência de linha; NOT EXISTS é seguro para ausência de correspondência.
- Subconsulta escalar deve retornar uma única linha e uma única coluna.

14. Lista de questões autorais - padrão FGV

Resolva as 20 questões sem consultar o gabarito. As questões foram construídas com formatos variados: conceito direto, caso prático, leitura de tabela, interpretação de consulta, afirmativas e relacione. O gabarito foi definido organicamente, sem quota fixa de letras.

Questão 1

Em um sistema de acompanhamento de processos, uma consulta deve listar o número do processo, a classe processual e a data de distribuição de todos os processos cadastrados, sem aplicar filtro, junção ou agregação. A instrução SQL mais adequada é:

- (A) SELECT * FROM processo WHERE numero, classe, data_distribuicao;
- (B) SELECT numero, classe, data_distribuicao WHERE processo;
- (C) SELECT numero, classe, data_distribuicao FROM processo;
- (D) FROM processo SELECT numero, classe, data_distribuicao;
- (E) SELECT processo(numero, classe, data_distribuicao);

Questão 2

Considere a necessidade de retornar os nomes das unidades judiciárias e a quantidade de processos ativos em cada uma, exibindo apenas unidades com mais de 100 processos ativos. Assinale a consulta que melhor atende ao requisito.

Tabelas:
 UNIDADE(id_unidade, nome)
 PROCESSO(id_processo, id_unidade, situacao)

- (A) SELECT u.nome, COUNT(*) total FROM unidade u JOIN processo p ON p.id_unidade = u.id_unidade GROUP BY u.nome WHERE p.situacao = 'ATIVO' AND COUNT(*) > 100;
- (B) SELECT u.nome, COUNT(*) total FROM unidade u JOIN processo p ON p.id_unidade = u.id_unidade WHERE COUNT(*) > 100 AND p.situacao = 'ATIVO' GROUP BY u.nome;
- (C) SELECT u.nome, p.situacao, COUNT(*) total FROM unidade u JOIN processo p ON p.id_unidade = u.id_unidade WHERE p.situacao = 'ATIVO' HAVING COUNT(*) > 100;
- (D) SELECT u.nome, COUNT(*) AS total FROM unidade u JOIN processo p ON p.id_unidade = u.id_unidade WHERE p.situacao = 'ATIVO' GROUP BY u.nome HAVING COUNT(*) > 100;
- (E) SELECT u.nome, COUNT(*) total FROM unidade u LEFT JOIN processo p ON p.id_unidade = u.id_unidade HAVING p.situacao = 'ATIVO' AND COUNT(*) > 100 ORDER BY u.nome;

Questão 3

No SQL, a diferença mais relevante entre WHERE e HAVING, para resolver questões de agregação, é que:

- (A) WHERE só pode ser usado com JOIN, enquanto HAVING só pode ser usado em subconsultas.
- (B) WHERE filtra linhas antes da agregação, enquanto HAVING filtra grupos após a agregação.
- (C) WHERE filtra colunas projetadas no SELECT, enquanto HAVING ordena os grupos.
- (D) WHERE é exclusivo de SQL Server, enquanto HAVING é exclusivo de PostgreSQL e Oracle.
- (E) WHERE substitui GROUP BY quando há função agregada no SELECT.

Questão 4

Considere as tabelas a seguir. Qual consulta retorna os processos que ainda não possuem audiência registrada?

```
PROCESSO(id_processo, numero)
AUDIENCIA(id_audiencia, id_processo, data_audiencia)
```

- (A) SELECT p.numero FROM processo p JOIN audiencia a ON a.id_processo = p.id_processo;
- (B) SELECT p.numero FROM processo p WHERE a.id_audiencia IS NULL;
- (C) SELECT p.numero FROM processo p LEFT JOIN audiencia a ON a.id_processo = p.id_processo WHERE p.id_processo IS NULL;
- (D) SELECT p.numero FROM processo p RIGHT JOIN audiencia a ON a.id_processo = p.id_processo WHERE a.id_audiencia IS NULL;
- (E) SELECT p.numero FROM processo p LEFT JOIN audiencia a ON a.id_processo = p.id_processo WHERE a.id_audiencia IS NULL;

Questão 5

Análise as afirmativas sobre NULL em SQL. I. A comparação coluna = NULL é a forma padrão de testar ausência de valor. II. COUNT(coluna) ignora valores NULL na coluna. III. WHERE status <> CANCELADO não garante a inclusão de linhas em que status seja NULL. Está correto o que se afirma em:

- (A) II e III, apenas.
- (B) I, apenas.
- (C) I e II, apenas.
- (D) I e III, apenas.
- (E) I, II e III.

Questão 6

Em uma consulta SQL padrão, considere a seguinte instrução. Assinale a avaliação correta.

```
SELECT id_unidade, nome, COUNT(*) AS total
FROM processo
GROUP BY id_unidade;
```

- (A) A consulta é sempre válida, porque qualquer coluna de uma tabela agrupada pode aparecer no SELECT.
- (B) A consulta é válida apenas se COUNT(*) for substituído por COUNT(nome).
- (C) A consulta tende a ser inválida em SQL relacional estrito, pois nome aparece no SELECT sem agregação e sem constar no GROUP BY.
- (D) A consulta só seria válida se ORDER BY nome fosse acrescentado.
- (E) A consulta está incorreta porque GROUP BY não pode ser usado com COUNT(*)

Questão 7

Relacione as cláusulas às funções em uma consulta SQL. 1. WHERE 2. GROUP BY 3. HAVING 4. ORDER BY. () Agrupa linhas por valores comuns. () Filtra linhas antes da agregação. () Ordena o resultado final. () Filtra grupos após a agregação. A sequência correta é:

- (A) 1, 2, 4, 3.
- (B) 2, 3, 4, 1.
- (C) 3, 1, 2, 4.
- (D) 4, 1, 3, 2.
- (E) 2, 1, 4, 3.

Questão 8

Considere os registros de PROCESSO abaixo e a consulta apresentada. Quantas linhas serão retornadas?

```
PROCESSO
id | classe | situacao
1 | Cível | ATIVO
2 | Cível | BAIXADO
3 | Penal | ATIVO
4 | Penal | NULL

SELECT classe
FROM processo
WHERE situacao <> 'BAIXADO';
```

- (A) 1 linha.

- (B) 2 linhas.
- (C) 3 linhas.
- (D) 2 linhas: Cível e Penal referentes aos registros com situação ATIVO.
- (E) 4 linhas, pois NULL é diferente de BAIXADO.

Questão 9

Em relação ao ORDER BY, assinale a afirmativa correta.

- (A) A ausência de ORDER BY garante ordenação pela chave primária.
- (B) ORDER BY define a ordenação do resultado final, e sua ausência impede pressupor ordem lógica das linhas retornadas.
- (C) ORDER BY sempre é executado antes de WHERE.
- (D) ORDER BY só pode ordenar colunas que não aparecem no SELECT.
- (E) ORDER BY substitui GROUP BY quando há agregação.

Questão 10

Em uma tabela MOVIMENTACAO(id_mov, id_processo, data_mov), deseja-se contar todas as movimentações por processo, inclusive linhas em que data_mov esteja nula. A expressão adequada é:

- (A) COUNT(*)
- (B) COUNT(data_mov) apenas, pois conta linhas nulas e não nulas.
- (C) SUM(data_mov)
- (D) AVG(id_mov)
- (E) COUNT(NULL)

Questão 11

Observe a consulta. Ela pretende retornar unidades com mais de 50 processos. Qual é o principal erro?

```
SELECT id_unidade, COUNT(*) AS total
FROM processo
WHERE COUNT(*) > 50
GROUP BY id_unidade;
```

- (A) A consulta deveria usar DISTINCT no lugar de GROUP BY.
- (B) A consulta deveria substituir COUNT(*) por SUM(*).
- (C) A consulta deveria remover id_unidade do SELECT.
- (D) A consulta deveria ordenar por total antes de agrupar.
- (E) A condição sobre COUNT(*) deve ficar em HAVING, não em WHERE.

Questão 12

A consulta abaixo retorna quais clientes?

```
CLIENTE(id_cliente, nome)
PEDIDO(id_pedido, id_cliente, valor)

SELECT c.nome
FROM cliente c
WHERE EXISTS (
  SELECT 1
  FROM pedido p
  WHERE p.id_cliente = c.id_cliente
);
```

- (A) Todos os clientes, pois SELECT 1 sempre retorna valor.
- (B) Apenas clientes sem pedido, pois EXISTS testa ausência de linhas.
- (C) Clientes para os quais existe pelo menos um pedido associado.
- (D) Apenas clientes cujo id_cliente seja igual a 1.
- (E) Apenas pedidos cujo cliente não exista.

Questão 13

A respeito de subconsultas, analise as afirmativas. I. Uma subconsulta escalar usada com = deve retornar, em regra, um único valor. II. Uma subconsulta correlacionada pode referenciar coluna da consulta externa. III. EXISTS depende da existência de linhas retornadas, não do valor específico projetado no SELECT interno. Está correto o que se afirma em:

- (A) I, apenas.

- (B) II, apenas.
- (C) I e II, apenas.
- (D) I, II e III.
- (E) II e III, apenas.

Questão 14

Considere que PARTE(id_parte, nome) possui 10 linhas e PROCESSO(id_processo, numero) possui 20 linhas. A consulta abaixo, sem condição de junção, tende a retornar quantas combinações?

```
SELECT *  
FROM parte, processo;
```

- (A) 200 combinações.
- (B) 30 combinações.
- (C) 20 combinações.
- (D) 10 combinações.
- (E) 0 combinações, pois a consulta é sempre inválida.

Questão 15

Em uma consulta com LIKE, deseja-se localizar nomes iniciados por Ana, como Ana, Ana Paula e Anastacio. O padrão mais adequado é:

- (A) LIKE '_Ana'
- (B) LIKE 'Ana%'
- (C) LIKE '%Ana'
- (D) LIKE 'Ana_'
- (E) LIKE '%Ana%' apenas se o nome tiver exatamente três letras.

Questão 16

Uma consulta usa LEFT JOIN para listar todos os processos e, quando houver, suas audiências marcadas. Porém, o analista acrescenta WHERE a.situacao = MARCADA. O efeito provável é:

- (A) Preservar todos os processos, inclusive sem audiência, porque LEFT JOIN sempre impede filtros.
- (B) Retornar apenas audiências sem processo.
- (C) Duplicar todos os processos, independentemente de audiência.
- (D) Transformar a coluna situacao em chave estrangeira.
- (E) Eliminar processos sem audiência marcada, fazendo o resultado se aproximar de um INNER JOIN para esse critério.

Questão 17

Em um relatório, deseja-se obter o total gasto por cliente em pedidos entregues, ordenando do maior para o menor total. Qual consulta está semanticamente mais adequada?

```
CLIENTE(id_cliente, nome)  
PEDIDO(id_pedido, id_cliente, valor_total, status)
```

- (A) SELECT c.nome, p.valor_total AS total FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente WHERE p.status = 'ENTREGUE' GROUP BY c.nome ORDER BY total DESC;
- (B) SELECT c.nome, SUM(p.valor_total) AS total FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente WHERE SUM(p.valor_total) > 0 AND p.status = 'ENTREGUE' GROUP BY c.nome ORDER BY total DESC;
- (C) SELECT c.nome, SUM(p.valor_total) AS total FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente WHERE p.status = 'ENTREGUE' GROUP BY c.id_cliente, c.nome ORDER BY total DESC;
- (D) SELECT nome, valor_total AS total FROM pedido WHERE status = 'ENTREGUE' ORDER BY total DESC;
- (E) SELECT c.nome, COUNT(p.valor_total) AS total FROM cliente c JOIN pedido p ON p.id_cliente = c.id_cliente GROUP BY c.nome ORDER BY total DESC;

Questão 18

A cláusula DISTINCT, em uma consulta SELECT, tem por finalidade:

- (A) Ordenar linhas duplicadas ao final da consulta.
- (B) Eliminar da tabela física todos os registros repetidos.
- (C) Agrupar linhas e permitir uso de HAVING.
- (D) Remover duplicidades do resultado projetado pela lista do SELECT.

(E) Substituir chaves primárias em tabelas sem integridade de entidade.

Questão 19

Sobre BETWEEN em SQL, assinale a afirmativa correta para o uso usual em consultas relacionais.

- (A) É exclusivo para campos textuais.
- (B) Inclui os limites inferior e superior da faixa.
- (C) Exclui sempre o limite superior.
- (D) Só pode ser usado depois de GROUP BY.
- (E) Substitui LIKE em qualquer busca textual.

Questão 20

Considere a consulta abaixo. Qual alternativa descreve melhor o papel da subconsulta derivada no FROM?

```
SELECT x.id_unidade, x.total
FROM (
  SELECT id_unidade, COUNT(*) AS total
  FROM processo
  GROUP BY id_unidade
) x
WHERE x.total > 100;
```

- (A) A subconsulta apaga da tabela PROCESSO as unidades com até 100 processos.
- (B) A subconsulta só seria válida se tivesse ORDER BY antes do GROUP BY.
- (C) A subconsulta cria uma tabela derivada temporária lógica com o total por unidade, que depois é filtrada pela consulta externa.
- (D) A subconsulta é equivalente a uma chave estrangeira entre PROCESSO e UNIDADE.
- (E) A subconsulta impede o uso de COUNT(*), pois agregações só podem ocorrer na consulta externa.

15. Gabarito resumido

1-C	2-D	3-B	4-E	5-A
6-C	7-E	8-D	9-B	10-A
11-E	12-C	13-D	14-A	15-B
16-E	17-C	18-D	19-B	20-C

16. Comentários completos

Questão 1 - Gabarito: C

A alternativa C está correta porque SELECT projeta as colunas desejadas e FROM indica a tabela de origem. A parece plausível por usar WHERE, mas WHERE não lista colunas; filtra linhas por predicado. B inverte a estrutura e não informa FROM corretamente. D troca a ordem sintática básica. E usa forma de chamada de função, não sintaxe SQL de consulta.

Questão 2 - Gabarito: D

D é correta: filtra linhas ativas no WHERE, agrupa por unidade e filtra grupos no HAVING. A seduz por conter todos os termos, mas WHERE vem depois de GROUP BY. B erra ao usar COUNT(*) no WHERE. C seleciona p.situacao sem agrupá-la e não possui GROUP BY adequado. E coloca condição de linha no HAVING e usa LEFT JOIN sem necessidade para o requisito.

Questão 3 - Gabarito: B

B é correta e resume a distinção que mais cai. A é falsa: ambos podem aparecer sem JOIN e HAVING não é exclusivo de subconsulta. C confunde filtro de linha com projeção e ordenação. D inventa exclusividade por SGBD. E é tentadora por mencionar agregação, mas WHERE não substitui GROUP BY.

Questão 4 - Gabarito: E

E é correta: LEFT JOIN preserva PROCESSO e WHERE a.id_audiencia IS NULL identifica ausência na tabela da direita. A retorna processos com audiência, não sem audiência. B referencia alias a sem declarar a tabela. C testa NULL na chave do processo preservado, o que não identifica ausência de audiência. D preserva AUDIENCIA, o lado errado para encontrar processos sem audiência.

Questão 5 - Gabarito: A

A é correta. II é verdadeira porque COUNT(coluna) conta valores não nulos. III é verdadeira porque comparação com NULL resulta desconhecida, não verdadeira. I é falsa: o teste correto é IS NULL. As demais alternativas incluem I ou omitem uma afirmação verdadeira.

Questão 6 - Gabarito: C

C é correta pela regra segura: colunas não agregadas no SELECT devem constar no GROUP BY. A é a pegadinha comum. B troca uma agregação por outra sem resolver a coluna nome. D confunde agrupamento com ordenação. E é falsa: GROUP BY com COUNT(*) é uso típico.

Questão 7 - Gabarito: E

E é correta: GROUP BY agrupa, WHERE filtra linhas, ORDER BY ordena e HAVING filtra grupos. A inverte WHERE e GROUP BY na primeira lacuna. B trata HAVING como filtro de linhas. C embaralha quase todos os conceitos. D atribui ordenação ao agrupamento.

Questão 8 - Gabarito: D

D é correta: retornam id 1 e id 3; o registro BAIXADO sai, e o NULL não satisfaz situacao <> BAIXADO. A erra a contagem. B acerta número mas não descreve, sendo menos precisa que D. C inclui indevidamente NULL. E trata NULL como valor comum.

Questão 9 - Gabarito: B

B é correta. Sem ORDER BY, não se deve presumir ordem. A é a armadilha de prática empírica. C inverte a ordem lógica. D é falsa: pode ordenar por colunas selecionadas e, conforme SGBD, por expressões não projetadas. E confunde ordenação com agrupamento.

Questão 10 - Gabarito: A

A é correta porque COUNT(*) conta linhas. B é tentadora, mas COUNT(data_mov) ignora data_mov NULL. C e D são agregações numéricas incompatíveis com a finalidade. E não conta linhas da tabela; COUNT de expressão NULL constante resulta zero em agregações usuais.

Questão 11 - Gabarito: E

E é correta: função agregada que filtra grupos deve ficar em HAVING. A confunde deduplicação com agregação. B propõe função inválida ou sem sentido. C remover id_unidade não atende ao relatório. D coloca ORDER BY como requisito inexistente e antes do agrupamento.

Questão 12 - Gabarito: C

C é correta: EXISTS verifica se a subconsulta correlacionada encontra ao menos um pedido do cliente corrente. A ignora a correlação e a necessidade de linhas retornadas. B descreve NOT EXISTS. D confunde SELECT 1 com filtro de id. E inverte a direção da consulta.

Questão 13 - Gabarito: D

D é correta. I está certa porque comparação escalar exige valor único. II define subconsulta correlacionada. III explica EXISTS. A, B, C e E omitem afirmações corretas.

Questão 14 - Gabarito: A

A é correta: sem condição, duas tabelas no FROM geram produto cartesiano, $10 \times 20 = 200$. B soma linhas, erro comum. C e D consideram apenas uma tabela. E é falsa: a consulta pode ser sintaticamente válida, embora geralmente indesejada.

Questão 15 - Gabarito: B

B é correta: Ana% significa começar por Ana e depois qualquer sequência. A exige um caractere antes de Ana. C localiza valores que terminam com Ana. D exige exatamente um caractere após Ana. E interpreta % de forma errada; % representa qualquer sequência, inclusive vazia.

Questão 16 - Gabarito: E

E é correta. O WHERE exige valor da tabela da direita; linhas sem correspondência têm NULL e são eliminadas. A trata LEFT JOIN como blindagem absoluta, o que é falso. B descreve outro objetivo. C não decorre do filtro. D mistura filtro com modelagem.

Questão 17 - Gabarito: C

C é correta: soma valores dos pedidos entregues por cliente e ordena pelo total. A seleciona valor individual p.valor_total sem agregá-lo. B usa SUM no WHERE. D não acessa nome do cliente e não agrega. E conta pedidos/valores, não soma total gasto, e ainda não filtra entregues.

Questão 18 - Gabarito: D

D é correta. DISTINCT age no resultado da consulta, sobre as expressões projetadas. A confunde DISTINCT com ORDER BY. B sugere alteração física na tabela, o que SELECT não faz. C confunde com GROUP BY. E mistura consulta com modelagem.

Questão 19 - Gabarito: B

B é correta: BETWEEN é inclusivo nos limites. A é falsa, pois também é usado com números e datas. C descreve uma convenção comum em intervalos de programação, mas não o BETWEEN usual. D é falsa. E confunde faixa com padrão textual.

Questão 20 - Gabarito: C

C é correta: a subconsulta no FROM produz uma tabela derivada lógica com id_unidade e total; a consulta externa filtra total > 100. A confunde SELECT com DELETE. B inventa obrigatoriedade de ORDER BY. D confunde consulta com integridade referencial. E é falsa: agregações podem ocorrer na subconsulta.

17. Referências consultadas

As fontes abaixo foram usadas para calibrar escopo, terminologia, profundidade e padrão de cobrança. As questões são autorais e não reproduzem questões reais literalmente.

- TJSC - Edital n. 10/2026, retificado em 10/04/2026, com o conteúdo de Banco de Dados e Engenharia de Dados.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- FGV Conhecimento - provas públicas correlatas com SQL, PostgreSQL, álgebra relacional, JOIN, LIMIT/OFFSET, DISTINCT e subconsultas.
- PostgreSQL Documentation - SELECT e Table Expressions.
- Microsoft Learn - SELECT, GROUP BY, HAVING, ORDER BY e Subqueries no SQL Server.
- MySQL Reference Manual - SELECT, JOIN, subqueries e EXISTS.
- Oracle Database SQL Language Reference - SELECT e subquery.

18. Checklist final da aula

- Se a questão pede filtro de linha, pense em WHERE.
- Se a questão pede filtro de grupo, pense em HAVING.
- Se a questão pede registros sem correspondência, pense em LEFT JOIN ... IS NULL ou NOT EXISTS.
- Se a questão pede relatório por categoria, pense em GROUP BY + função agregada.
- Se a questão traz NULL, não use comparação comum; pense em IS NULL, IS NOT NULL e lógica desconhecida.
- Se a questão cobra resultado ordenado, procure ORDER BY.
- Se a questão traz subconsulta com ausência, cuidado com NOT IN quando houver possibilidade de NULL.